

Projet 7 : Détection d'anomalies dans les transactions financières avec PySpark

Réalisé avec PySpark dans Google Colab

Table des matières

1. Introduction
 2. Contexte et problématique
 3. Description du jeu de données
 4. Analyse exploratoire et déséquilibre des classes
 5. Prétraitement des données
 6. Implémentation de l'Isolation Forest
 7. Implémentation de K-Means
 8. Classification supervisée avec Logistic Regression
 9. Évaluation des performances
 10. Analyse des erreurs et impact métier
 11. Stratégie de seuil optimal
 12. Comparaison des méthodes
 13. Recommandations et perspectives
 14. Conclusion
-

1. Introduction

La fraude bancaire représente aujourd'hui un enjeu majeur pour les institutions financières. Avec l'augmentation massive des transactions électroniques et des paiements en ligne, les banques doivent être capables de détecter rapidement les comportements suspects afin de limiter les pertes financières et protéger les clients.

Les systèmes traditionnels basés uniquement sur des règles fixes deviennent insuffisants face à des techniques de fraude de plus en plus sophistiquées. Pour cette raison, les approches de Machine Learning sont largement utilisées dans les systèmes modernes de détection de fraude.

L'objectif principal de ce projet est de développer un système de détection d'anomalies appliqué aux transactions financières en utilisant PySpark dans l'environnement Google Colab. Plusieurs approches ont été étudiées afin de comparer leurs performances dans un contexte de données fortement déséquilibrées.

Les méthodes implémentées dans ce projet sont :

- Isolation Forest
- K-Means Clustering
- Logistic Regression

Le projet met particulièrement l'accent sur :

- Le traitement des données déséquilibrées
 - L'évaluation des modèles avec des métriques adaptées
 - L'analyse métier des erreurs
 - La comparaison entre méthodes supervisées et non supervisées
-

2. Contexte et problématique

Dans un système bancaire réel, le volume de transactions traitées quotidiennement est extrêmement élevé. Parmi ces transactions, seules quelques-unes sont frauduleuses.

Ce déséquilibre crée une difficulté importante pour les algorithmes de Machine Learning. En effet, un modèle peut atteindre une très grande accuracy en prédisant uniquement la classe majoritaire, tout en échouant complètement à détecter les fraudes.

Le principal défi du projet est donc de construire un système capable de détecter efficacement les transactions frauduleuses malgré leur rareté.

Dans un contexte métier :

- Une fraude non détectée entraîne une perte financière directe pour la banque.
- Une transaction normale détectée comme fraude peut gêner le client mais reste généralement acceptable.

Ainsi, dans les systèmes de détection de fraude, le rappel devient une métrique particulièrement importante.

3. Description du jeu de données

Le dataset utilisé contient des transactions financières anonymisées.

Chaque ligne représente une transaction bancaire et contient plusieurs variables numériques représentant différentes caractéristiques de la transaction.

La colonne cible est :

- Class = 0 : transaction normale
- Class = 1 : transaction frauduleuse

Le dataset contient :

- 154 283 transactions normales
- 338 transactions frauduleuses

Le nombre total de transactions est donc :

154 621 transactions.

Le pourcentage de fraude est très faible :

0.22 % seulement.

Cette caractéristique rend le problème particulièrement difficile.

4. Analyse exploratoire et déséquilibre des classes

Une première étape importante du projet a consisté à explorer le dataset afin de comprendre sa structure.

Les principales observations sont les suivantes :

- Les données sont entièrement numériques.
- Aucune valeur manquante importante n'a été observée.
- Les classes sont extrêmement déséquilibrées.

La distribution des classes est la suivante :

Classe	Nombre
Normal (0)	154 283
Fraude (1)	338

Le ratio entre transactions normales et frauduleuses est d'environ :

456 transactions normales pour 1 transaction frauduleuse.

Cette situation crée plusieurs problèmes :

- Les modèles peuvent ignorer la classe minoritaire.
- L'accuracy devient trompeuse.
- Les faux négatifs deviennent critiques.

Pour cette raison, des techniques spécifiques de gestion du déséquilibre ont été nécessaires.

5. Prétraitement des données

Avant l'entraînement des modèles, plusieurs étapes de prétraitement ont été réalisées.

5.1 Assemblage des variables

Les algorithmes de PySpark ML nécessitent que toutes les variables explicatives soient regroupées dans une seule colonne vectorielle.

Pour cela, la classe `VectorAssembler` a été utilisée.

Toutes les colonnes sauf la variable cible ont été regroupées dans une colonne appelée : features.

5.2 Normalisation des données

Les algorithmes comme K-Means et Logistic Regression sont sensibles à l'échelle des variables.

Une normalisation a donc été appliquée avec StandardScaler.

Cette étape permet :

- d'éviter qu'une variable domine les autres,
- d'améliorer la convergence,
- d'obtenir de meilleures performances.

Les données finales utilisées par les modèles contiennent :

- scaledFeatures : variables normalisées
- Class : variable cible

5.3 Séparation entraînement / test

Le dataset a ensuite été séparé en deux parties :

- 80 % pour l'entraînement
- 20 % pour le test

Cette séparation permet d'évaluer les performances des modèles sur des données non vues.

6. Implémentation de l'Isolation Forest

L'Isolation Forest est une méthode non supervisée de détection d'anomalies.

Contrairement aux modèles supervisés, elle ne nécessite pas l'utilisation des labels lors de l'apprentissage.

Le principe de l'algorithme est d'isoler les observations rares et différentes du comportement normal.

Dans ce projet, Isolation Forest a été implémenté avec la bibliothèque scikit-learn après conversion du dataset Spark vers Pandas.

Le paramètre contamination a été fixé à :

0.002

Cette valeur correspond approximativement au taux réel de fraude dans le dataset.

6.1 Résultats obtenus

Les résultats de la matrice de confusion sont :

Classe réelle	Prédiction	Nombre
Fraude détectée correctement	1	128
Fraude non détectée	0	210
Faux positifs	1	182
Transactions normales correctes	0	154101

6.2 Analyse des performances

Le modèle a réussi à détecter une partie des fraudes mais présente un rappel relativement faible.

Le recall est d'environ 38 %.

Cela signifie que le modèle ne détecte qu'une partie limitée des fraudes.

Malgré cette limitation, Isolation Forest présente plusieurs avantages :

- Détection d'anomalies inconnues
- Aucune nécessité d'étiquettes
- Adaptation aux nouveaux types de fraude

Cependant, pour un système bancaire réel, les performances restent insuffisantes.

7. Implémentation de K-Means

K-Means est un algorithme de clustering permettant de regrouper les données selon leur similarité.

Dans ce projet, le nombre de clusters a été fixé à :

$k = 2$

L'objectif était d'observer si les transactions frauduleuses se regroupent naturellement dans un cluster spécifique.

7.1 Résultats obtenus

La distribution obtenue est la suivante :

Cluster	Classe	Nombre
1	Normal	111804
1	Fraude	90
0	Normal	42479
0	Fraude	248

7.2 Analyse des clusters

Le cluster 0 contient une proportion plus importante de fraudes.

Le taux de fraude dans le cluster 0 est supérieur à celui du cluster 1.

Cela montre que K-Means a réussi à regrouper partiellement certaines transactions frauduleuses.

Cependant, les fraudes restent fortement mélangées avec les transactions normales.

Les principales limites observées sont :

- Sensibilité au déséquilibre des classes
- Mauvaise séparation des données rares
- Difficulté à détecter précisément les anomalies

K-Means peut être utile comme outil d'exploration mais reste insuffisant pour une détection précise de fraude.

8. Classification supervisée avec Logistic Regression

La régression logistique constitue l'approche supervisée principale de ce projet.

Contrairement aux méthodes précédentes, elle utilise les labels réels pour apprendre à distinguer les transactions normales des transactions frauduleuses.

8.1 Gestion du déséquilibre des classes

Afin de résoudre le problème du déséquilibre, une stratégie de pondération des classes a été appliquée.

Le ratio calculé est :

456.46

Cela signifie qu'une transaction frauduleuse reçoit un poids environ 456 fois supérieur à celui d'une transaction normale.

Cette stratégie permet de :

- Donner plus d'importance aux fraudes
- Réduire les faux négatifs
- Améliorer le rappel

8.2 Entraînement du modèle

Le modèle Logistic Regression a été entraîné sur les données normalisées.

Le dataset d'entraînement a été utilisé pour ajuster les paramètres du modèle.

Le dataset de test a ensuite permis d'évaluer les performances sur des données non utilisées pendant l'apprentissage.

9. Évaluation des performances

Plusieurs métriques adaptées aux données déséquilibrées ont été utilisées.

9.1 AUC-ROC

Le modèle a obtenu :

AUC-ROC = 0.9576

Cette valeur indique une excellente capacité de séparation entre les transactions normales et frauduleuses.

Une valeur proche de 1 signifie que le modèle distingue très bien les deux classes.

9.2 Precision

Precision = 0.9997

Cela signifie que presque toutes les transactions identifiées comme fraude sont réellement frauduleuses.

Le nombre de faux positifs reste relativement limité.

9.3 Recall

Recall = 0.9838

Cette métrique est particulièrement importante.

Le modèle détecte environ 98.4 % des fraudes.

Cela représente une très bonne performance pour un système bancaire.

9.4 F1-Score

F1-score = 0.9899

Le F1-score montre un excellent équilibre entre precision et recall.

10. Analyse des erreurs et impact métier

La matrice de confusion obtenue est :

Classe réelle	Prédiction	Nombre
Fraude non détectée	0	10
Transactions normales correctes	0	30456
Fraudes détectées	1	58
Faux positifs	1	503

10.1 Faux négatifs

Les faux négatifs représentent les transactions frauduleuses que le système n'a pas réussi à détecter.

Dans le cadre de ce projet, le modèle de régression logistique a produit seulement 10 faux négatifs.

Même si ce nombre reste relativement faible par rapport au volume total des transactions, ces erreurs sont les plus critiques dans un système bancaire.

En effet, lorsqu'une fraude n'est pas détectée :

- la banque peut subir une perte financière directe,
- le client peut perdre confiance dans le système,
- des problèmes juridiques ou réglementaires peuvent apparaître,
- l'image de l'établissement financier peut être dégradée.

Dans les systèmes de détection de fraude, la réduction des faux négatifs constitue donc une priorité absolue.

Le recall élevé obtenu dans ce projet montre que le modèle est capable de détecter la majorité des transactions frauduleuses.

10.2 Faux positifs

Les faux positifs correspondent aux transactions normales identifiées comme frauduleuses.

Dans ce projet, le modèle a produit 503 faux positifs.

Ces erreurs sont moins graves que les faux négatifs mais peuvent néanmoins avoir plusieurs conséquences :

- blocage temporaire d'une transaction légitime,
- gêne ou frustration du client,
- augmentation du nombre de vérifications manuelles,
- coût opérationnel supplémentaire pour la banque.

Cependant, dans le domaine bancaire, il est généralement préférable d'accepter un certain nombre de faux positifs afin de réduire fortement les risques de fraude réelle.

Le compromis entre faux positifs et faux négatifs dépend principalement des objectifs métier et du niveau de sécurité souhaité.

10.3 Analyse globale des erreurs

L'analyse des résultats montre que le modèle supervisé obtient un excellent équilibre entre sécurité et fiabilité.

Les principaux points observés sont :

- Très faible nombre de fraudes manquées,
- Très haute capacité de détection,
- Nombre raisonnable de faux positifs compte tenu du contexte.

Ces résultats démontrent que le modèle est adapté à une utilisation dans un système de surveillance des transactions financières.

11. Stratégie de seuil optimal

Par défaut, la régression logistique utilise un seuil de classification de 0.5.

Cela signifie qu'une transaction est considérée comme frauduleuse lorsque la probabilité prédite dépasse 50 %.

Cependant, dans les systèmes de fraude bancaire, le choix du seuil influence directement les performances du modèle.

11.1 Impact du seuil sur les performances

Lorsque le seuil est diminué :

- davantage de transactions sont classées comme fraude,
- le recall augmente,
- le nombre de faux positifs augmente également.

À l'inverse, lorsque le seuil est augmenté :

- la précision peut augmenter,
- mais davantage de fraudes risquent d'être ignorées.

Le choix du seuil représente donc un compromis entre sécurité et confort utilisateur.

11.2 Choix du seuil dans ce projet

Dans ce projet, la priorité est donnée à la réduction des faux négatifs.

Pour cette raison, un seuil relativement bas peut être préférable afin de maximiser le rappel.

Une stratégie possible consiste à tester plusieurs seuils différents :

Seuil	Recall	Precision
0.5	Élevé	Très élevé
0.4	Plus élevé	Légèrement réduit
0.3	Très élevé	Plus faible

Cette approche permet d'adapter le système selon les besoins de la banque.

11.3 Importance métier du seuil

Le seuil optimal dépend du coût des erreurs.

Dans un environnement où la fraude coûte très cher, il est préférable de privilégier un recall élevé.

Dans un environnement où l'expérience utilisateur est prioritaire, un seuil plus élevé peut être utilisé afin de réduire les faux positifs.

12. Comparaison des méthodes

Les trois approches implémentées dans ce projet présentent des caractéristiques différentes.

12.1 Tableau comparatif

Méthode	Type	Avantages	Inconvénients	Performance
Isolation Forest	Non supervisé	Détection d'anomalies inconnues	Recall faible	Moyenne
K-Means	Non supervisé	Simple et rapide	Séparation limitée	Faible
Logistic Regression	Supervisé	Excellente précision et recall	Besoin de labels	Excellente

12.2 Analyse comparative détaillée

Isolation Forest

Isolation Forest est particulièrement utile pour détecter des comportements inhabituels sans avoir besoin d'exemples de fraude étiquetés.

Cette méthode peut être intéressante dans des situations où les types de fraude changent fréquemment.

Cependant, les performances obtenues montrent que cette approche reste insuffisante seule pour une utilisation critique.

K-Means

K-Means permet de regrouper les transactions similaires.

Le modèle a réussi à identifier un cluster contenant une proportion plus élevée de transactions frauduleuses.

Néanmoins, la séparation reste faible à cause du fort déséquilibre des données.

Logistic Regression

La régression logistique supervisée a obtenu les meilleurs résultats.

Grâce à l'utilisation des labels et à la gestion du déséquilibre par pondération, le modèle atteint un excellent recall ainsi qu'une très forte précision.

Cette méthode apparaît donc comme la solution la plus adaptée pour ce dataset.

13. Recommandations et perspectives

À partir des résultats obtenus, plusieurs recommandations peuvent être formulées.

13.1 Utilisation en production

La régression logistique pondérée constitue le meilleur choix pour un environnement réel.

Le modèle présente :

- une très bonne stabilité,
- un excellent rappel,
- une forte précision,
- des performances adaptées aux systèmes bancaires.

13.2 Mise en place d'un système hybride

Une combinaison de méthodes supervisées et non supervisées pourrait améliorer davantage les performances.

Par exemple :

- Logistic Regression pour détecter les fraudes connues,
- Isolation Forest pour détecter de nouveaux comportements suspects.

Cette architecture hybride est souvent utilisée dans les systèmes de sécurité modernes.

13.3 Réentraînement régulier

Les comportements frauduleux évoluent constamment.

Il est donc nécessaire de mettre à jour régulièrement les modèles avec de nouvelles données afin de conserver de bonnes performances.

13.4 Perspectives futures

Plusieurs pistes d'amélioration peuvent être envisagées :

- utilisation de Random Forest,
- implémentation de XGBoost,
- modèles de Deep Learning,
- analyse temps réel avec Spark Streaming,
- détection comportementale avancée.

Ces approches pourraient permettre d'améliorer encore la précision du système.

14. Conclusion

Ce projet a permis d'étudier plusieurs techniques de Machine Learning appliquées à la détection de fraude bancaire.

Le dataset utilisé présente un déséquilibre extrême avec seulement 0.22 % de transactions frauduleuses.

Cette caractéristique rend la détection particulièrement complexe.

Trois approches principales ont été comparées :

- Isolation Forest,
- K-Means,
- Logistic Regression.

Les méthodes non supervisées ont montré certaines capacités de détection d'anomalies mais restent limitées dans un contexte fortement déséquilibré.

La Logistic Regression avec gestion du déséquilibre a obtenu les meilleures performances.

Les résultats obtenus sont excellents :

- AUC-ROC élevé,
- Recall supérieur à 98 %,
- Très forte précision,
- Excellent F1-score.

L'analyse métier montre également que la réduction des faux négatifs constitue la priorité principale dans un système bancaire.

En conclusion, PySpark représente une solution puissante pour l'analyse de grandes quantités de données financières et la mise en œuvre de systèmes de détection de fraude performants.

15. Détails techniques de l'implémentation

15.1 Environnement de travail

Le projet a été réalisé dans l'environnement Google Colab.

Les principales technologies utilisées sont :

Technologie	Rôle
Python	Langage principal

Technologie	Rôle
PySpark	Traitement distribué des données
Spark MLlib	Algorithmes de Machine Learning
scikit-learn	Isolation Forest
Google Colab	Environnement d'exécution

L'utilisation de PySpark permet de traiter efficacement de grands volumes de données grâce au calcul distribué.

15.2 Architecture du pipeline

Le pipeline global du projet suit les étapes suivantes :

1. Chargement du dataset
2. Exploration des données
3. Analyse du déséquilibre des classes
4. Prétraitement et normalisation
5. Implémentation des modèles non supervisés
6. Implémentation du modèle supervisé
7. Évaluation des performances
8. Analyse métier des erreurs
9. Comparaison des approches

Cette organisation permet de structurer clairement le processus de détection de fraude.

15.3 Préparation des features

Toutes les variables numériques ont été regroupées dans une seule colonne vectorielle grâce à VectorAssembler.

Exemple de code utilisé :

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=features,
    outputCol="features"
)
```

Les données ont ensuite été normalisées avec StandardScaler.

```
from pyspark.ml.feature import StandardScaler

scaler = StandardScaler(
    inputCol="features",
```

```
        outputCol="scaledFeatures"  
    )
```

15.4 Gestion du déséquilibre

Le déséquilibre des classes constitue l'un des principaux défis du projet.

Une stratégie de pondération des classes a été utilisée afin d'augmenter l'importance des transactions frauduleuses pendant l'entraînement.

Exemple de code :

```
from pyspark.sql.functions import when  
  
df_weighted = df_final.withColumn(  
    "weight",  
    when(df_final.Class == 1, balance_ratio).otherwise(1.0)  
)
```

Cette approche permet d'améliorer fortement le recall du modèle.

15.5 Évaluation des modèles

Les modèles ont été évalués avec plusieurs métriques adaptées aux données déséquilibrées :

- Precision
- Recall
- F1-score
- AUC-ROC

Le choix de ces métriques est important car l'accuracy seule serait trompeuse dans ce contexte.

16. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées pendant la réalisation du projet.

16.1 Compatibilité d'Isolation Forest avec PySpark

PySpark ne propose pas directement une implémentation native stable d'Isolation Forest dans l'environnement Google Colab.

Pour contourner cette limitation, les données ont été converties vers Pandas afin d'utiliser la bibliothèque scikit-learn.

Cette solution reste valide dans le cadre du projet.

16.2 Déséquilibre extrême des classes

Le très faible nombre de fraudes rend l'apprentissage difficile.

Sans gestion du déséquilibre :

- les modèles ignorent la classe minoritaire,
- le recall devient très faible,
- les performances sont trompeuses.

La pondération des classes a permis de résoudre une grande partie de ce problème.

16.3 Temps de calcul

Certaines opérations de Machine Learning peuvent devenir coûteuses lorsque le volume de données augmente.

L'utilisation de Spark permet de réduire ce problème grâce au traitement distribué.

17. Apports pédagogiques du projet

Ce projet a permis de développer plusieurs compétences importantes dans le domaine de la Data Science et du Big Data.

17.1 Utilisation de PySpark

Le projet a permis de comprendre :

- le fonctionnement de Spark,
- la manipulation des DataFrames Spark,
- l'utilisation de Spark MLlib,
- le traitement de grands volumes de données.

17.2 Machine Learning appliqué

Plusieurs concepts importants ont été étudiés :

- classification supervisée,
- clustering,
- détection d'anomalies,
- gestion des données déséquilibrées,
- évaluation des modèles.

17.3 Analyse métier

Le projet a également montré l'importance de l'analyse métier dans les systèmes de fraude.

Un modèle performant doit non seulement obtenir de bonnes métriques mais aussi répondre aux besoins réels de l'entreprise.

18. Conclusion générale

La détection de fraude bancaire constitue un problème complexe nécessitant des méthodes avancées de Machine Learning.

Dans ce projet, plusieurs approches ont été étudiées et comparées afin d'identifier la solution la plus adaptée.

Les résultats montrent clairement que les approches supervisées avec gestion du déséquilibre offrent les meilleures performances.

La Logistic Regression pondérée a obtenu :

- une excellente capacité de détection,
- un recall très élevé,
- une forte précision,
- un excellent compromis global.

Les méthodes non supervisées restent intéressantes pour détecter de nouveaux comportements suspects mais ne suffisent pas seules pour une utilisation critique.

Le projet démontre également l'importance des technologies Big Data comme Spark dans l'analyse de données financières massives.

Enfin, cette étude confirme que la combinaison du Machine Learning et de l'analyse métier constitue aujourd'hui une solution essentielle pour lutter efficacement contre la fraude bancaire.

Bibliographie

- Documentation officielle Apache Spark
- Documentation PySpark MLlib
- Documentation scikit-learn
- Documentation Google Colab

- Articles académiques sur la détection de fraude bancaire

Les faux négatifs représentent les fraudes non détectées.

Dans ce projet, seulement 10 transactions frauduleuses n'ont pas été détectées.

Ces erreurs sont les plus graves car elles entraînent des pertes financières directes.

10.2 Faux positifs

Les faux positifs correspondent à des transactions normales détectées comme fraude.

Le modèle produit 503 faux positifs.

Bien que cela puisse gêner certains clients, cette situation reste généralement acceptable dans le secteur bancaire.

10.3 Interprétation métier

Dans un système de détection de fraude :

- Un faux négatif coûte de l'argent à la banque.
- Un faux positif crée principalement une gêne temporaire.

Pour cette raison, les banques privilégient généralement un recall élevé.

11. Stratégie de seuil optimal

Par défaut, Logistic Regression utilise un seuil de classification égal à 0.5.

Cependant, ce seuil peut être ajusté afin d'améliorer les performances selon les objectifs métier.

11.1 Diminution du seuil

Si le seuil est diminué :

- davantage de fraudes seront détectées,
- le recall augmentera,
- mais les faux positifs seront plus nombreux.

11.2 Augmentation du seuil

Si le seuil est augmenté :

- le nombre de faux positifs diminuera,

- mais davantage de fraudes risquent de ne pas être détectées.

11.3 Choix du seuil optimal

Dans le cadre de ce projet, il est préférable d'utiliser un seuil relativement bas afin de maximiser la détection des fraudes.

Le choix du seuil dépend donc du compromis métier entre :

- sécurité financière,
 - satisfaction client,
 - coût des erreurs.
-

12. Comparaison des méthodes

Méthode	Type	Avantages	Limites	Performance
Isolation Forest	Non supervisé	Détecte anomalies inconnues	Recall faible	Moyenne
K-Means	Non supervisé	Simple et rapide	Mauvaise séparation	Faible
Logistic Regression	Supervisé	Très haute précision et recall	Besoin de labels	Excellente

Analyse comparative

Les méthodes non supervisées permettent de détecter certains comportements anormaux sans utiliser les labels.

Cependant, leurs performances restent limitées dans un contexte fortement déséquilibré.

La régression logistique supervisée offre les meilleurs résultats grâce à l'utilisation des labels et à la gestion du déséquilibre.

13. Recommandations et perspectives

À partir des résultats obtenus, plusieurs recommandations peuvent être proposées.

13.1 Utilisation d'un modèle supervisé

La Logistic Regression avec pondération des classes constitue la meilleure solution pour un environnement de production.

13.2 Approche hybride

Il peut être intéressant de combiner :

- un modèle supervisé,
- et une méthode de détection d'anomalies.

Cette stratégie permettrait de détecter à la fois :

- les fraudes connues,
- les nouveaux comportements suspects.

13.3 Surveillance continue

Les comportements frauduleux évoluent rapidement.

Il est donc nécessaire de réentraîner régulièrement les modèles avec de nouvelles données.

13.4 Perspectives futures

Plusieurs améliorations pourraient être étudiées :

- Random Forest
 - XGBoost
 - Réseaux de neurones
 - Deep Learning
 - Streaming temps réel avec Spark Streaming
-

14. Conclusion

Ce projet a permis d'étudier plusieurs techniques de détection de fraude appliquées à des transactions financières.

Le dataset utilisé présente un déséquilibre extrême, ce qui rend la détection des fraudes particulièrement complexe.

Trois approches ont été comparées :

- Isolation Forest

- K-Means
- Logistic Regression

Les résultats montrent que :

- les méthodes non supervisées peuvent détecter certaines anomalies,
- mais leurs performances restent limitées,
- tandis que la Logistic Regression pondérée offre des performances excellentes.

Le modèle supervisé a obtenu :

- un AUC-ROC très élevé,
- un excellent recall,
- un très bon F1-score.

Le projet met également en évidence l'importance de l'analyse métier dans les systèmes de détection de fraude.

Dans un contexte bancaire, la réduction des faux négatifs reste l'objectif principal afin de limiter les pertes financières.

En conclusion, la combinaison de PySpark et des techniques de Machine Learning constitue une solution puissante pour la détection d'anomalies financières à grande échelle.

Bibliographie

- Documentation officielle Apache Spark
- Documentation PySpark MLlib
- Documentation scikit-learn
- Articles académiques sur la détection de fraude bancaire
- Documentation Google Colab